

Design a fraud detection System for banking transaction using java, spring-boot , angular, sql and open shift. The system should identify suspicious transactions in real time.

Real-Time Fraud Detection System for Banking Transactions

Technology Stack: Java, Spring Boot, Angular, SQL, OpenShift

1. Business Requirements

The system should:

- Process banking transactions in real time.
- Detect suspicious transactions within milliseconds.
- Block or flag high-risk transactions.
- Generate alerts for fraud analysts.
- Maintain audit trails.
- Support regulatory compliance.
- Scale to millions of transactions daily.

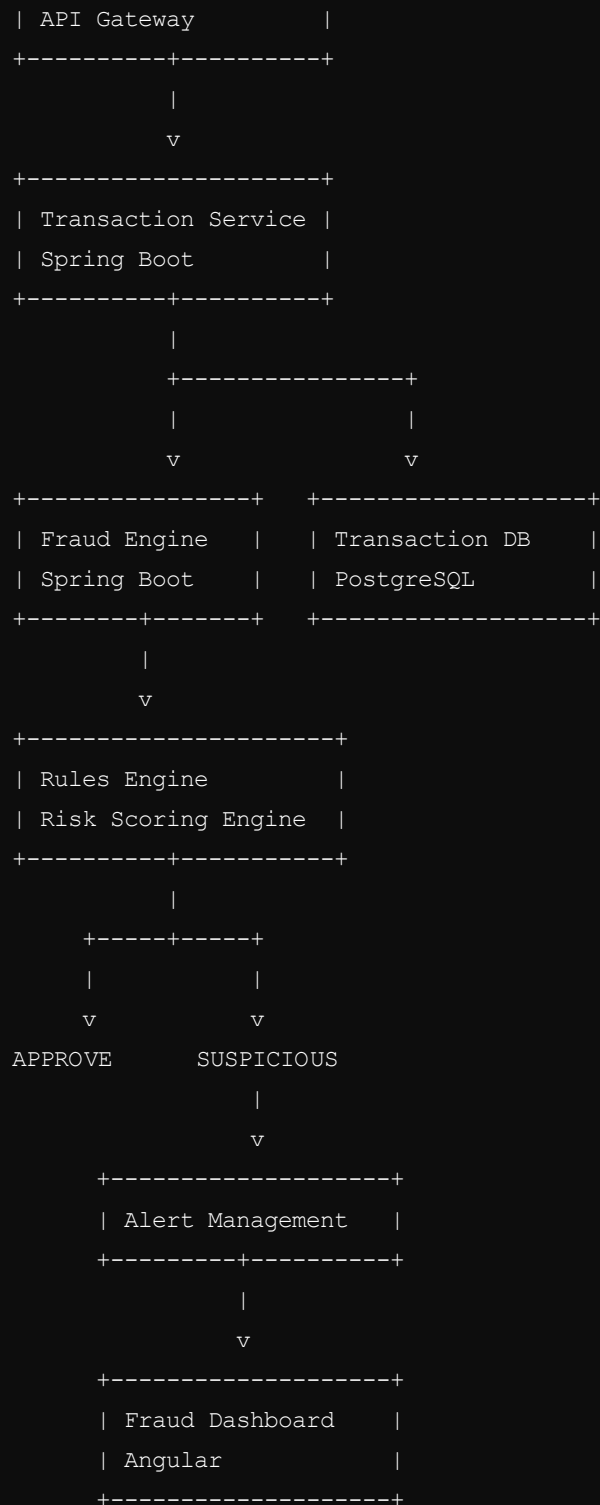
Fraud Examples

- Multiple transactions in a short period.
- Transactions from unusual locations.
- Sudden high-value transactions.
- Impossible travel (India → USA within minutes).
- New device usage.
- Blacklisted accounts/cards.
- Velocity attacks (many transactions rapidly).

2. High-Level Architecture

```
+-----+
| Mobile/Web Banking |
+-----+-----+
|
| v
+-----+
```





3. Microservices Architecture

Transaction Service

Responsible for:

- Receive transaction requests
- Validate account
- Persist transaction
- Call Fraud Service

Endpoints:

 http



POST /api/transactions

GET /api/transactions/{id}

Fraud Detection Service

Responsible for:

- Real-time fraud checks
- Risk score calculation
- Rule execution
- Alert generation

Endpoints:

 http



POST /api/fraud/check

GET /api/fraud/alerts

Notification Service

Responsible for:

- SMS alerts
- Email alerts
- Push notifications

User Profile Service

Stores:

- Customer behavior
- Device information
- Historical transaction patterns

4. Database Design (SQL)

CUSTOMER

</> SQL



```
CREATE TABLE CUSTOMER(  
    customer_id BIGINT PRIMARY KEY,  
    name VARCHAR(100),  
    mobile VARCHAR(20),  
    risk_level VARCHAR(20)  
);
```

ACCOUNT

</> SQL



```
CREATE TABLE ACCOUNT(  
    account_id BIGINT PRIMARY KEY,  
    customer_id BIGINT,  
    balance DECIMAL(18,2),  
    status VARCHAR(20)  
);
```

TRANSACTION

</> SQL



```
CREATE TABLE TRANSACTION(  
    transaction_id BIGINT PRIMARY KEY,  
    account_id BIGINT,  
    amount DECIMAL(18,2),  
    transaction_time TIMESTAMP,  
    merchant VARCHAR(100),  
    location VARCHAR(100),  
    device_id VARCHAR(100),  
    status VARCHAR(20)  
);
```

FRAUD_ALERT

</> SQL



```
CREATE TABLE FRAUD_ALERT(
    alert_id BIGINT PRIMARY KEY,
    transaction_id BIGINT,
    risk_score INTEGER,
    alert_type VARCHAR(50),
    created_time TIMESTAMP,
    status VARCHAR(20)
);
```

DEVICE_HISTORY

</> SQL



```
CREATE TABLE DEVICE_HISTORY(
    device_id VARCHAR(100),
    customer_id BIGINT,
    first_seen TIMESTAMP
);
```

5. Fraud Detection Flow

```
Transaction Received
    |
    v
Validate Account
    |
    v
Load Customer Profile
    |
    v
Execute Fraud Rules
    |
    v
Calculate Risk Score
    |
    +-----+
    |               |
Score < 50      Score >= 50
    |               |
    v               v
  Approve        Create Alert
                  |
                  v
              Block / Review
```



6. Rule-Based Fraud Detection

Rule 1: High Amount

</> Java



```
if(transaction.getAmount() > 100000) {  
    riskScore += 40;  
}
```

Rule 2: New Device

</> Java



```
if(!deviceRepository.exists(deviceId)) {  
    riskScore += 20;  
}
```

Rule 3: Multiple Transactions

</> Java



```
long count =  
transactionRepository  
.findLast10Minutes(customerId);  
  
if(count > 5) {  
    riskScore += 30;  
}
```

Rule 4: Blacklisted Merchant

</> Java



```
if(blackList.contains(merchant)) {  
    riskScore += 50;  
}
```

Rule 5: Geo Location Anomaly

</> Java



```
if(isDifferentCountry(customer,lastLocation,currentLocation)){  
    riskScore += 25;  
}
```

7. Fraud Scoring Engine

Risk Calculation

</> Java



```
public Integer calculateRisk(Transaction txn){  
  
    int score = 0;  
  
    score += amountRule(txn);  
    score += velocityRule(txn);  
    score += locationRule(txn);  
    score += deviceRule(txn);  
  
    return score;  
}
```

Decision Matrix

Risk Score	Action
0-30	Approve
31-60	Flag
61-100	Block
>100	Immediate Fraud Alert

8. Spring Boot Design

Controller

</> Java



```
@RestController
@RequestMapping("/api/fraud")
public class FraudController {

    @PostMapping("/check")
    public FraudResponse check(
        @RequestBody Transaction txn) {

        return fraudService.analyze(txn);
    }
}
```

Service

</> Java



```
@Service
public class FraudService {

    public FraudResponse analyze(
        Transaction transaction){

        int riskScore =
            riskEngine.calculate(transaction);

        if(riskScore > 60){
            createAlert(transaction);
        }

        return buildResponse(riskScore);
    }
}
```

JPA Repository

</> Java



```
@Repository
public interface TransactionRepository
    extends JpaRepository<Transaction, Long> {

}
```


9. Angular Dashboard

Features

- Live fraud alerts
- Transaction monitoring
- Fraud analytics
- Risk score visualization
- Investigation workflow

Screens

Dashboard

```
+-----+
| Fraud Alerts Today: 145 |
| High Risk: 23          |
| Blocked: 12            |
+-----+
```



Alert Management

```
Transaction ID
Customer
Risk Score
Status
Action
```



Investigation Screen

```
Customer History
Device History
Geo Location
Transaction Pattern
```



10. OpenShift Deployment

Pods

```
fraud-service
transaction-service
notification-service
angular-ui
postgresql
```



Deployment YAML

YAML



```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: fraud-service
spec:
  replicas: 3
  selector:
    matchLabels:
      app: fraud-service
  template:
    metadata:
      labels:
        app: fraud-service
    spec:
      containers:
        - name: fraud-service
          image: fraud-service:1.0
          ports:
            - containerPort: 8080
```

OpenShift Features Used

- Horizontal Pod Autoscaler
- Service Mesh
- Route Exposure
- ConfigMaps
- Secrets
- Persistent Volumes
- Health Checks

11. Security Design

Authentication

- OAuth2
- JWT Tokens

Authorization

```
ADMIN
FRAUD_ANALYST
CUSTOMER_SERVICE
AUDITOR
```



Encryption

- TLS 1.3
- Database encryption
- Tokenized card data

12. Production-Grade Enhancements

For a real banking system, add:

Event Streaming

```
Transaction Service
  |
  v
Kafka
  |
  v
Fraud Detection Service
```



Benefits:

- Real-time processing
- High throughput
- Fault tolerance

Machine Learning Layer

Models:

- Isolation Forest
- XGBoost
- Random Forest

- Neural Networks

Features:

```
Transaction Amount
Time
Device
Location
Merchant
Customer Behavior
```



ML score can be combined with rule score:

```
Final Risk =
70% Rule Score +
30% ML Score
```



Enterprise Architecture Recommendation

```
Angular UI
|
OpenShift Route
|
API Gateway
|
+-----+
| Transaction Service |
+-----+
      |
      v
    Kafka
      |
      v
+-----+
| Fraud Detection      |
| Rule Engine + ML     |
+-----+
      |
      +-----+
      |         |
      v         v
PostgreSQL  Alert Service
      |
      v
Fraud Dashboard
```

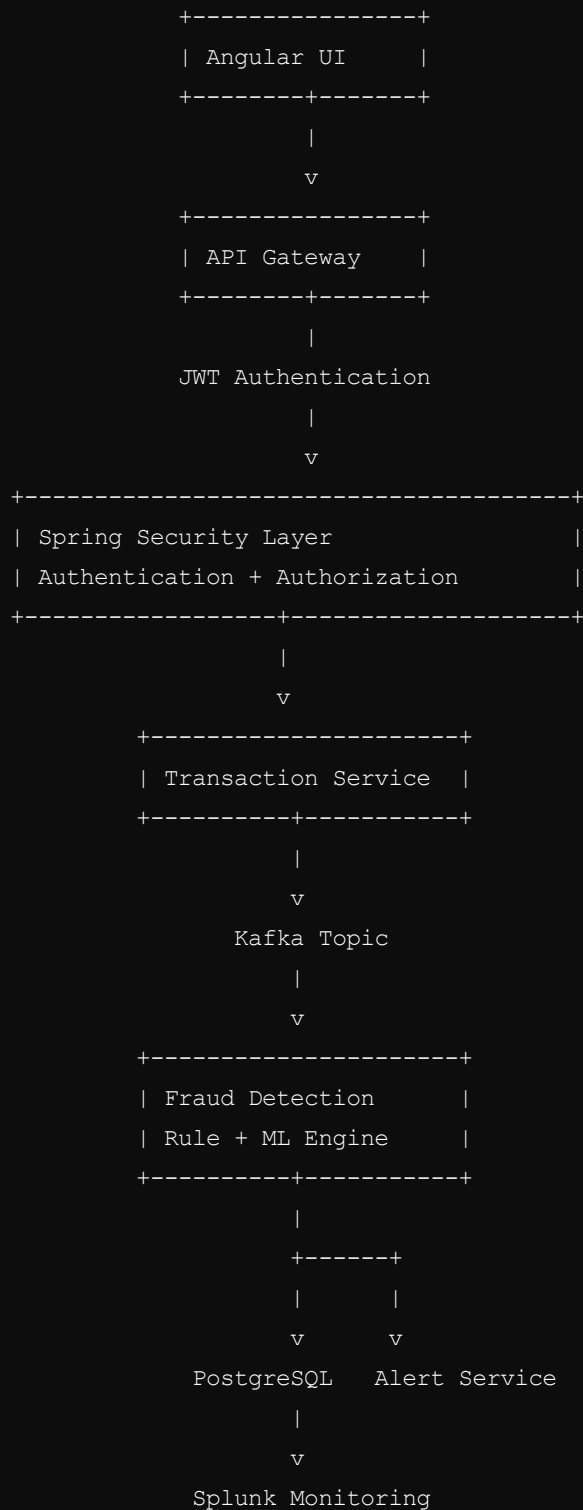


This architecture is scalable, cloud-native, event-driven, and suitable for a large bank processing thousands of transactions per second while detecting fraud in near real time.

include JWT, security and monitor user logs in splunk

Adding **JWT Security**, **Spring Security**, and **Splunk-based monitoring** makes the solution enterprise-grade and suitable for banking environments.

Enhanced Fraud Detection Architecture



Security Architecture

Authentication Flow

```
User Login
  |
  v
Authentication Service
  |
  +--> Validate User
  |
  +--> Generate JWT
  |
  v
Return Access Token
```



JWT Structure

JSON



```
{
  "sub": "customer123",
  "role": "CUSTOMER",
  "iat": 1718000000,
  "exp": 1718003600
}
```

Spring Security Implementation

Dependencies

XML



```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
</dependency>
```

JWT Utility

</> Java



```
@Component
public class JwtUtil {

    private final String SECRET =
        "BankingFraudSecretKey";

    public String generateToken(String username){

        return Jwts.builder()
            .setSubject(username)
            .setIssuedAt(new Date())
            .setExpiration(
                new Date(System.currentTimeMillis()
                    + 3600000))
            .signWith(
                SignatureAlgorithm.HS256,
                SECRET)
            .compact();
    }
}
```

JWT Filter

</> Java



```
@Component
public class JwtAuthenticationFilter
    extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain filterChain)
        throws ServletException, IOException {

        String authHeader =
            request.getHeader("Authorization");

        if(authHeader != null &&
            authHeader.startsWith("Bearer ")) {

            String token =
                authHeader.substring(7);
        }
    }
}
```

```
        // Validate JWT
    }

    filterChain.doFilter(request, response);
}
}
```

Security Configuration

Java



```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain securityFilterChain(
        HttpSecurity http) throws Exception {

        return http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(
                sm -> sm.sessionCreationPolicy(
                    SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/auth/**")
                .permitAll()
                .requestMatchers("/admin/**")
                .hasRole("ADMIN")
                .requestMatchers("/fraud/**")
                .hasRole("FRAUD_ANALYST")
                .anyRequest()
                .authenticated())
            .build();
    }
}
```

Role-Based Access Control (RBAC)

Roles

```
CUSTOMER
BANK_OPERATOR
FRAUD_ANALYST
```



Authorization Matrix

API	Customer	Analyst	Admin
Transactions	Yes	Yes	Yes
Fraud Alerts	No	Yes	Yes
User Management	No	No	Yes
Audit Logs	No	Yes	Yes

Audit Logging Strategy

Every user action must be audited.

Audit Events

- Login
- Logout
- Transaction Creation
- Transaction Approval
- Fraud Alert Review
- Password Change
- User Creation
- Role Change

Audit Entity

</> SQL

```
CREATE TABLE AUDIT_LOG(  
  audit_id BIGSERIAL PRIMARY KEY,  
  user_id VARCHAR(100),  
  action VARCHAR(100),  
  resource VARCHAR(100),  
  ip_address VARCHAR(50),  
  event_time TIMESTAMP,  
  status VARCHAR(20)  
);
```

Audit Aspect (AOP)

</> Java



```
@Aspect
@Component
public class AuditAspect {

    @AfterReturning(
        pointcut="@annotation(Auditable)")
    public void logAudit(
        JoinPoint joinPoint){

        AuditLog log =
            new AuditLog();

        log.setAction(
            joinPoint.getSignature().getName());

        auditRepository.save(log);
    }
}
```

Transaction Monitoring in Splunk

Why Splunk?

Monitor:

- User login activity
- Failed logins
- Fraud alerts
- API calls
- Suspicious transactions
- Unauthorized access attempts
- System health

Log Structure

Spring Boot outputs JSON logs.

</> JSON



```
{
  "timestamp": "2026-06-10T12:10:30",
  "service": "fraud-service",
  "userId": "cust123",
  "transactionId": "TXN1001",
  "eventType": "FRAUD_ALERT",
  "riskScore": 85,
  "ipAddress": "10.10.1.20"
}
```

Logback Configuration

</> XML



```
<appender name="SPLUNK"
  class="com.splunk.logging.HttpEventCollectorLogbackAppender">

  <url>
    http://splunk-hec:8088/services/collector
  </url>

  <token>
    ${SPLUNK_TOKEN}
  </token>

  <source>fraud-service</source>

</appender>
```

Fraud Events Sent to Splunk

</> Java



```
logger.info(
  "FRAUD_ALERT user={} txn={} score={}",
  customerId,
  transactionId,
  riskScore);
```

Splunk Dashboards

Fraud Dashboard

High Risk Transactions
Blocked Transactions
Fraud Alerts By Region
Fraud Alerts By Merchant
Risk Trend Analysis



Security Dashboard

Failed Login Attempts
JWT Validation Failures
Unauthorized API Access
Role Escalation Attempts
Account Lockouts



User Activity Dashboard

Top Active Users
Transactions Per User
Login Locations
Device Usage
Session Duration



Example Splunk Queries (SPL)

Fraud Alerts

```
</> spl  
  
index=banking eventType=FRAUD_ALERT  
| stats count by riskScore
```



Failed Logins

```
</> spl  
  
index=banking eventType=LOGIN_FAILED  
| stats count by userId
```



Suspicious IPs

 spl



```
index=banking
| stats count by ipAddress
| where count > 100
```

Multiple Transactions Within 5 Minutes

 spl



```
index=banking eventType=TRANSACTION
| bucket _time span=5m
| stats count by userId,_time
| where count > 10
```

OpenShift Deployment Enhancements

Secrets

 YAML



```
apiVersion: v1
kind: Secret
metadata:
  name: jwt-secret
type: Opaque
data:
  JWT_SECRET: xxxxxx
```

ConfigMap

 YAML



```
apiVersion: v1
kind: ConfigMap
metadata:
  name: fraud-config
data:
  SPLUNK_URL: http://splunk-hec:8088
```

Banking Compliance Features

PCI-DSS

- Card tokenization
- Encrypted storage
- Audit trails

GDPR

- User consent tracking
- Data retention policies

RBI Guidelines

- Transaction monitoring
- Fraud detection controls
- Audit logging
- Access management

Enterprise Production Architecture

```
Angular UI
  |
API Gateway
  |
JWT + Spring Security
  |
Transaction Service
  |
Kafka
  |
Fraud Detection Service
(Rules + ML + Risk Engine)
  |
PostgreSQL
  |
Audit Service
  |
Splunk HEC
  |
+-----+
| Splunk Dashboards |
| Fraud Monitoring  |
| Security Monitoring |
| User Activity Logs |
| Audit Reports     |
+-----+

OpenShift:
```



- 3 API Pods
- 3 Fraud Pods
- 3 Audit Pods
- PostgreSQL HA
- Kafka Cluster
- Splunk Forwarder
- HPA Autoscaling

This design provides **real-time fraud detection, JWT-based authentication, Spring Security authorization, complete audit trails, and centralized Splunk observability**, which aligns well with enterprise banking requirements.